Unsupervised Vector Quantization, Expectation-Maximization Limits, and Discrete Partitioning

A Unified Mathematical, Information-Theoretic, and Physical Foundation of K-Means and K-Medoids

---

Contents

1 The Partitioning Clustering Paradigm

Unsupervised partitioning algorithms segment an unlabelled dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)}\} \subset \mathbb{R}^d$ into a pre-specified number $K \in \mathbb{N}^+$ of disjoint, non-empty clusters $\mathcal{C} = \{C_1, C_2, \dots, C_K\}$.

Definition 1.1 (**Hard Cluster Partition**). A collection of subsets $\mathcal{C} = \{C_1, \dots, C_K\}$ constitutes a valid hard partition of $\mathcal{D}$ if and only if it satisfies three structural constraints:

1. **Non-Emptiness:** $C_k \neq \emptyset \quad \forall k \in \{1, 2, \dots, K\}$.
2. **Exhaustive Coverage:** $\bigcup_{k=1}^{K} C_k = \mathcal{D}$.
3. **Mutual Exclusivity:** $C_k \cap C_l = \emptyset \quad \forall k \neq l$.

In hard clustering, partition membership is represented by a binary indicator matrix $\mathbf{R} \in \{0,1\}^{N \times K}$, where element $r_{ik} = 1$ if instance $\mathbf{x}^{(i)} \in C_k$, and $r_{ik} = 0$ otherwise. The exclusivity constraint enforces $\sum_{k=1}^{K} r_{ik} = 1$ for every observation i.

1.1 Centroid vs. Medoid Exemplar Representations

Partitioning models summarize each cluster C_k using a central prototype (exemplar):

1. **K-Means Prototype (Centroid $\boldsymbol{\mu}_k \in \mathbb{R}^d$):** The exemplar $\boldsymbol{\mu}_k$ is defined as the continuous arithmetic mean of all vectors assigned to cluster C_k. The centroid is generally a synthetic point that does not coincide with any observed data instance in $\mathcal{D}$.
2. **K-Medoids Prototype (Medoid $\mathbf{m}_k \in \mathcal{D}$):** The exemplar $\mathbf{m}_k$ is strictly constrained to be an actual observed data point from $\mathcal{D}$. It represents the centrally located instance that minimizes total dissimilarity to all other members of cluster C_k.

1.2 Primal Objective Functions

1. K-Means Objective: Within-Cluster Sum of Squares (WCSS / Inertia)

K-Means evaluates structural distortion using the squared Euclidean metric (L_2^2 norm):

$$J_{\text{KMeans}}(\mathbf{R}, \boldsymbol{\mu}) = \sum_{i=1}^{N} \sum_{k=1}^{K} r_{ik} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_k \right\|_2^2 = \sum_{k=1}^{K} \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|_2^2 \tag{1}$$

2. K-Medoids Objective: Total Absolute Dissimilarity Cost

K-Medoids generalizes exemplar clustering to arbitrary dissimilarity metrics $d(\cdot, \cdot)$ (such as L_1 Manhattan, Cosine, or Jaccard distances):

$$J_{\text{KMedoids}}(\mathbf{R}, \mathbf{M}) = \sum_{i=1}^{N} \sum_{k=1}^{K} r_{ik}\, d\left(\mathbf{x}^{(i)}, \mathbf{m}_k\right), \quad \text{subject to } \mathbf{m}_k \in \mathcal{D} \quad \forall k \tag{2}$$

2 K-Means: Lloyd's Algorithm and Convergence Dynamics

Minimizing the WCSS objective J_{KMeans} globally across all possible assignments is an NP-hard combinatorial optimization problem ($\mathcal{O}(K^N)$ partitions). In practice, K-Means uses **Lloyd's Algorithm** (1957, 1982), an alternating block-coordinate descent scheme.

2.1 The Two-Step Iterative Optimization Scheme

Step 1: Cluster Assignment (Expectation Step)

Fixing the centroid matrix $\boldsymbol{\mu} = [\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K]$, we minimize J_{KMeans} with respect to assignment matrix $\mathbf{R}$. Because the objective decouples across samples, each observation $\mathbf{x}^{(i)}$ is independently assigned to its nearest centroid:

$$r_{ik}^{(t+1)} = \begin{cases} 1, & \text{if } k = \arg\min_{l \in \{1,\ldots,K\}} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_l^{(t)} \right\|_2^2 \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

Step 2: Centroid Update (Maximization Step)

Fixing assignment matrix $\mathbf{R}$, we minimize J_{KMeans} with respect to centroids $\boldsymbol{\mu}_k$. Setting the partial derivative with respect to $\boldsymbol{\mu}_k$ to zero:

$$\frac{\partial J_{\text{KMeans}}}{\partial \boldsymbol{\mu}_k} = \frac{\partial}{\partial \boldsymbol{\mu}_k} \sum_{i=1}^{N} r_{ik} \left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k \right)^T \left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k \right) = \mathbf{0} \tag{4}$$

$$-2 \sum_{i=1}^{N} r_{ik} \left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k \right) = \mathbf{0} \implies \sum_{i=1}^{N} r_{ik} \mathbf{x}^{(i)} = \boldsymbol{\mu}_k \sum_{i=1}^{N} r_{ik} \tag{5}$$

Defining $|C_k| = \sum_{i=1}^{N} r_{ik}$ as the cardinality of cluster C_k, the updated centroid is the exact arithmetic mean:

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}^{(i)} \tag{6}$$

2.2 Proof of Monotonic Convergence and Finite Termination

Theorem 2.1 (**Monotonicity and Termination of Lloyd's Algorithm**)**.** Lloyd's algorithm strictly decreases or maintains the WCSS objective J_{KMeans} at every iteration:

$$J_{\text{KMeans}} \left(\mathbf{R}^{(t+1)}, \boldsymbol{\mu}^{(t+1)} \right) \leq J_{\text{KMeans}} \left(\mathbf{R}^{(t+1)}, \boldsymbol{\mu}^{(t)} \right) \leq J_{\text{KMeans}} \left(\mathbf{R}^{(t)}, \boldsymbol{\mu}^{(t)} \right) \tag{7}$$

Furthermore, the algorithm converges to a local minimum in a finite number of iterations.

Proof. . **Assignment Step:** For a fixed $\boldsymbol{\mu}^{(t)}$, updating $r_{ik}^{(t+1)}$ assigns each sample to its closest centroid, ensuring $J\left(\mathbf{R}^{(t+1)}, \boldsymbol{\mu}^{(t)}\right) \leq J\left(\mathbf{R}^{(t)}, \boldsymbol{\mu}^{(t)}\right)$.

Update Step: For a fixed $\mathbf{R}^{(t+1)}$, setting $\boldsymbol{\mu}_k^{(t+1)}$ to the arithmetic mean minimizes the quadratic sum of squared Euclidean deviations for that partition, ensuring $J\left(\mathbf{R}^{(t+1)}, \boldsymbol{\mu}^{(t+1)}\right) \leq J\left(\mathbf{R}^{(t+1)}, \boldsymbol{\mu}^{(t)}\right)$.

Finite Termination: Because there exists a finite number of K^N distinct hard partitions of N points into K sets, and every strict update monotonically decreases J, the algorithm cannot revisit a previous partition state. Thus, it terminates at a stationary local minimum in finite steps. □

2.3 K-Means++ Smart Initialization

Random centroid initialization can cause Lloyd's algorithm to converge to poor local minima. The **K-Means++** initialization algorithm (Arthur and Vassilvitskii, 2007) uses randomized probability proportional to squared distance to place initial seeds:

. Choose first centroid $\boldsymbol{\mu}_1$ uniformly at random from dataset $\mathcal{D}$.

For each data point $\mathbf{x} \in \mathcal{D}$, compute shortest distance to already selected centroids:

$$D(\mathbf{x}) = \min_{l \in \{1,\ldots,j-1\}} \|\mathbf{x} - \boldsymbol{\mu}_l\|_2 \tag{8}$$

Select next centroid $\boldsymbol{\mu}_j$ randomly from $\mathcal{D}$ with probability $P(\mathbf{x})$:

$$P(\mathbf{x}) = \frac{D(\mathbf{x})^2}{\sum_{\mathbf{x}' \in \mathcal{D}} D(\mathbf{x}')^2} \tag{9}$$

Repeat steps 2–3 until K centroids are chosen.

Theorem 2.2 (K**-Means++ Approximation Guarantee**)**.** If initial centroids are selected via K-Means++, the expected WCSS of the final converged state satisfies an $\mathcal{O}(\log K)$ competitive bound against the global optimal clustering J^*:

$$\mathbb{E}\left[J_{\text{KMeans}}\right] \leq 8(\ln K + 2)J^* \tag{10}$$

3 K-Medoids: PAM Algorithm and Robust Dissimilarity

While K-Means is computationally fast ($\mathcal{O}(NKdi)$), it exhibits two major structural limitations: . **Sensitivity to Outliers:** Minimizing squared Euclidean distances (L_2^2) causes extreme noise vectors $\|\mathbf{x}_{\text{out}}\| \to \infty$ to heavily pull centroids away from true cluster cores.

Metric Restriction: Arithmetic means $\boldsymbol{\mu}_k = \frac{1}{|C_k|}\sum \mathbf{x}$ require continuous Euclidean vector spaces $\mathbb{R}^d$, making K-Means inapplicable to categorical, graph, or non-Euclidean distance structures.

K-Medoids addresses both issues by forcing prototypes to be actual data points $\mathbf{m}_k \in \mathcal{D}$, optimizing arbitrary metric or non-metric dissimilarities $d(\cdot, \cdot)$.

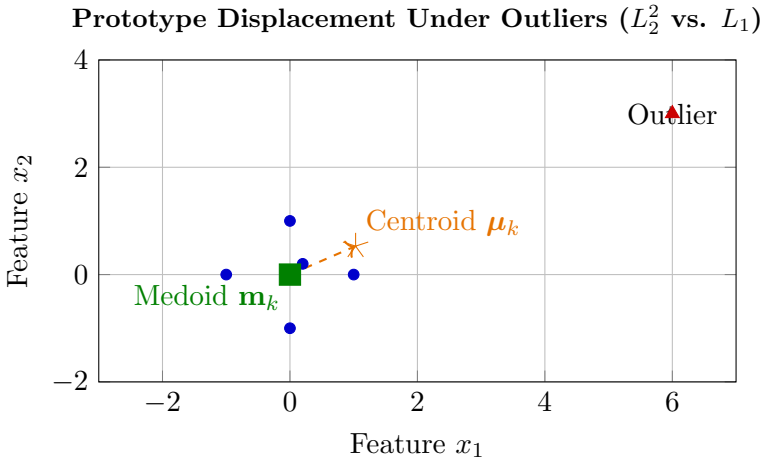

Figure 1: Impact of a single outlier on cluster prototypes. The K-Means centroid $\boldsymbol{\mu}_k$ is pulled significantly toward the outlier, whereas the K-Medoids prototype $\mathbf{m}_k$ remains stable at the true cluster median.

3.1 Partitioning Around Medoids (PAM) Algorithm

The standard exact algorithm for K-Medoids is **PAM (Partitioning Around Medoids)** (Kaufman and Rousseeuw, 1990), which operates in two phases:

Phase 1: BUILD Phase

Select K initial medoids sequentially. The first medoid $\mathbf{m}_1$ is chosen as the point minimizing total distance to all other data points. Each subsequent medoid $\mathbf{m}_j$ is chosen to maximize the reduction in total cost J_{KMedoids}.

Phase 2: SWAP Phase

Iteratively evaluate every pair of current medoid $\mathbf{m}_k \in \mathbf{M}$ and non-medoid candidate $\mathbf{o}_h \in \mathcal{D} \setminus \mathbf{M}$. Calculate the net effect on total cost ΔC_{kh} if $\mathbf{m}_k$ and $\mathbf{o}_h$ are swapped:

$$\Delta C_{kh} = J_{\text{KMedoids}}\left((\mathbf{M} \setminus \{\mathbf{m}_k\}) \cup \{\mathbf{o}_h\}\right) - J_{\text{KMedoids}}(\mathbf{M}) \tag{11}$$

If $\min_{k,h} \Delta C_{kh} < 0$, perform the swap that yields the largest cost reduction:

$$\mathbf{M} \leftarrow (\mathbf{M} \setminus \{\mathbf{m}_{k^*}\}) \cup \{\mathbf{o}_{h^*}\} \quad \text{where } (k^*, h^*) = \arg\min_{k,h} \Delta C_{kh} \tag{12}$$

Repeat the SWAP phase until $\Delta C_{kh} \geq 0$ for all candidate pairs.

3.2 Computational Complexity Comparison

Algorithm	Metric Constraint	Time Complexity (Per Iteration)	Sensitivity to Noise
K-**Means (Lloyd)**	Squared L_2 Euclidean	$\mathcal{O}(NKd)$	High (Squared penalty)
K-**Medoids (PAM)**	Arbitrary $d(\cdot,\cdot)$	$\mathcal{O}\left(K(N-K)^2\right)$	Low (Median prototype)
CLARA / CLARANS	Arbitrary $d(\cdot,\cdot)$	$\mathcal{O}\left(KS^2 + K(N-K)\right)$	Low (Subsampled PAM)

Table 1: Algorithmic comparison between K-Means and K-Medoids variants.

4 Information-Theoretic and Statistical Mechanics Views

4.1 Vector Quantization (VQ) and Rate-Distortion Theory

In signal processing and information theory, K-Means is equivalent to the **Lloyd-Max Quantizer** for lossy data compression.

A Vector Quantizer maps a continuous input space $\mathbb{R}^d$ to a discrete codebook $\mathcal{C} = \{\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K\}$ using a quantization function $Q : \mathbb{R}^d \to \mathcal{C}$.

According to **Rate-Distortion Theory** (Shannon, 1948), transmitting a quantized vector requires code rate $R = \log_2 K$ bits. The expected distortion D under squared error loss is:

$$D = \mathbb{E}\left[\|\mathbf{X} - Q(\mathbf{X})\|_2^2\right] = \int_{\mathbb{R}^d} \|\mathbf{x} - Q(\mathbf{x})\|_2^2 \, p(\mathbf{x}) \, d\mathbf{x} \tag{13}$$

K-Means minimizes empirical rate-distortion by placing codebook vectors $\boldsymbol{\mu}_k$ at regions of high data probability density.

4.2 K-Means as the Hard-Assignment Limit of Gaussian Mixture Models

Consider a **Gaussian Mixture Model (GMM)** with K components, equal isotropic covariances $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}_d$, and uniform priors $\pi_k = 1/K$:

$$p(\mathbf{x} \mid \boldsymbol{\theta}) = \sum_{k=1}^{K} \frac{1}{K} \frac{1}{(2\pi\sigma^2)^{d/2}} \exp\left(-\frac{\|\mathbf{x} - \boldsymbol{\mu}_k\|_2^2}{2\sigma^2}\right) \tag{14}$$

Using Bayes' Theorem, the soft posterior probability (responsibility) γ_{ik} that component k generated observation $\mathbf{x}^{(i)}$ is:

$$\gamma_{ik} = \frac{\exp\left(-\frac{\|\mathbf{x}^{(i)} - \boldsymbol{\mu}_k\|_2^2}{2\sigma^2}\right)}{\sum_{l=1}^{K} \exp\left(-\frac{\|\mathbf{x}^{(i)} - \boldsymbol{\mu}_l\|_2^2}{2\sigma^2}\right)} \tag{15}$$

Theorem 4.1 (**Zero-Variance Softmax Limit**)**.** As cluster variance approaches zero ($\sigma^2 \to 0^+$), the soft responsibilities γ_{ik} of a GMM converge to hard K-Means binary assignments $r_{ik} \in \{0, 1\}$:

$$\lim_{\sigma^2 \to 0^+} \gamma_{ik} = \begin{cases} 1, & \text{if } k = \arg\min_l \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_l\|_2^2 \\ 0, & \text{otherwise} \end{cases} \tag{16}$$

Proof. Divide the numerator and denominator of γ_{ik} by $\exp\left(-\frac{\min_l \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_l\|_2^2}{2\sigma^2}\right)$:

$$\gamma_{ik} = \frac{\exp\left(-\frac{\|\mathbf{x}^{(i)} - \boldsymbol{\mu}_k\|_2^2 - \min_l \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_l\|_2^2}{2\sigma^2}\right)}{\sum_{j=1}^{K} \exp\left(-\frac{\|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\|_2^2 - \min_l \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_l\|_2^2}{2\sigma^2}\right)} \tag{17}$$

If $k = \arg\min_l \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_l\|_2^2$, the exponential numerator evaluates to $e^0 = 1$. For any $j \neq k$, $\|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\|_2^2 - \min_l \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_l\|_2^2 > 0$. Taking the limit as $\sigma^2 \to 0^+$ causes all non-minimal terms in the denominator to vanish ($e^{-\infty} = 0$), yielding $\lim_{\sigma^2 \to 0^+} \gamma_{ik} = \frac{1}{1+0} = 1$. □

4.3 Deterministic Annealing and Free Energy Minimization

In statistical mechanics, clustering can be framed through **Deterministic Annealing** (Rose, 1998).

Assigning a data point $\mathbf{x}^{(i)}$ to prototype $\boldsymbol{\mu}_k$ incurs an energy cost $E_{ik} = \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_k\|_2^2$. At thermal equilibrium with temperature $T = 2\sigma^2$, the system minimizes the **Helmholtz Free Energy** F:

$$F = U - TS = -\frac{1}{\beta} \sum_{i=1}^{N} \ln \sum_{k=1}^{K} \exp\left(-\beta \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_k\|_2^2\right) \tag{18}$$

where $\beta = \frac{1}{T} = \frac{1}{2\sigma^2}$ is inverse temperature, U is internal energy, and S is Shannon entropy.

- **High Temperature ($T \to \infty$):** Entropy dominates. All assignments are uniform ($\gamma_{ik} = 1/K$), and all centroids collapse to the global dataset mean.
- **Low Temperature ($T \to 0^+$):** Energy dominates. The system undergoes phase transitions, bifurcating centroids to minimize U, converging to hard K-Means.

5 Physical Analogies: Center of Mass and Gravitational Wells

5.1 Center of Mass and Parallel Axis Theorem

The K-Means objective maps directly onto classical multi-body mechanics.

Consider a system of N particles in d-dimensional space, where each particle i has unit mass $m_i = 1$ and position vector $\mathbf{r}_i = \mathbf{x}^{(i)}$.

For a cluster C_k containing mass $M_k = |C_k|$, the **Center of Mass** $\mathbf{R}_{\text{cm}}^{(k)}$ is:

$$\mathbf{R}_{\text{cm}}^{(k)} = \frac{1}{M_k} \sum_{i \in C_k} m_i \mathbf{r}_i = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}^{(i)} = \boldsymbol{\mu}_k \tag{19}$$

The WCSS contribution of cluster C_k corresponds to its **Moment of Inertia** I_k about its center of mass:

$$I_k = \sum_{i \in C_k} m_i \left\|\mathbf{r}_i - \mathbf{R}_{\text{cm}}^{(k)}\right\|_2^2 = \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|_2^2 \tag{20}$$

Theorem 5.1 (**Mechanical Parallel Axis Theorem for Clusters**)**.** For any arbitrary reference point $\mathbf{a} \in \mathbb{R}^d$, the total inertia of cluster C_k about $\mathbf{a}$ equals its intrinsic inertia about its centroid $\boldsymbol{\mu}_k$ plus the orbital inertia of its total mass M_k concentrated at $\boldsymbol{\mu}_k$:

$$\sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \mathbf{a}\|_2^2 = \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|_2^2 + |C_k| \|\boldsymbol{\mu}_k - \mathbf{a}\|_2^2 \tag{21}$$

This theorem provides mechanical proof that setting centroid $\boldsymbol{\mu}_k$ to the center of mass strictly minimizes the cluster's moment of inertia.

5.2 Gravitational Potential Basins and Voronoi Quantization

K-Means constructs a multi-body potential energy field. Each centroid $\boldsymbol{\mu}_k$ acts as an attractive central force origin generating a parabolic potential well $V_k(\mathbf{x}) = \|\mathbf{x} - \boldsymbol{\mu}_k\|_2^2$.

Data particles $\mathbf{x}^{(i)}$ move to the lowest potential energy state, partitioning space into convex polyhedral **Voronoi Tessellations** bounded by perpendicular bisecting hyperplanes between adjacent centroids:

$$V_k = \left\{ \mathbf{x} \in \mathbb{R}^d \;\middle|\; \|\mathbf{x} - \boldsymbol{\mu}_k\|_2 \leq \|\mathbf{x} - \boldsymbol{\mu}_l\|_2 \quad \forall l \neq k \right\} \tag{22}$$

6 Production-Grade Vectorized NumPy Implementation

The following Python module provides vectorized implementations of both K-Means (with K-Means++ initialization) and K-Medoids (PAM algorithm with fast matrix swaps).

```python
import numpy as np

class KMeansEngine:
    """
    Production-grade K-Means Clustering Engine.
    Features K-Means++ initialization, vectorized Euclidean distance calculations,
    and monotonic WCSS (Inertia) optimization.
    """
    def __init__(self, n_clusters: int = 8, max_iter: int = 300, tol: float = 1e-4, init: str = 'k-means++'):
        self.n_clusters = n_clusters
        self.max_iter = max_iter
        self.tol = tol
        self.init = init

        self.cluster_centers_ = None  # Centroids (K, d)
        self.labels_ = None           # Cluster indices (N,)
        self.inertia_ = None          # Final WCSS scalar

    def _kmeans_plus_plus_init(self, X: np.ndarray) -> np.ndarray:
        """K-Means++ initialization for smart centroid placement."""
        N, d = X.shape
        centroids = np.zeros((self.n_clusters, d))

        # 1. Choose first centroid uniformly at random
        first_idx = np.random.choice(N)
        centroids[0] = X[first_idx]

        # 2. Distance matrix tracking shortest squared distance to any centroid
        min_dists_sq = np.full(N, np.inf)

        for k in range(1, self.n_clusters):
            # Compute squared distances to newest centroid
            prev_centroid = centroids[k - 1]
            dist_sq_new = np.sum((X - prev_centroid)**2, axis=1)
            min_dists_sq = np.minimum(min_dists_sq, dist_sq_new)

            # Sample next centroid with probability proportional to D(x)^2
            probs = min_dists_sq / np.sum(min_dists_sq)
            next_idx = np.random.choice(N, p=probs)
            centroids[k] = X[next_idx]

        return centroids
```

```
43
44      def _compute_squared_distances(self, X: np.ndarray, centroids: np.
    ndarray) -> np.ndarray:
45          """
46          Fast vectorized L2 squared distance calculation:
47          ||X - C||^2 = ||X||^2 + ||C||^2 - 2 * X @ C^T
48          """
49          X_sq = np.sum(X**2, axis=1, keepdims=True)          # (N, 1)
50          C_sq = np.sum(centroids**2, axis=1, keepdims=True).T # (1, K)
51          dot_prod = X @ centroids.T                           # (N, K)
52
53          dists_sq = X_sq + C_sq - 2.0 * dot_prod
54          return np.maximum(dists_sq, 0.0) # Guard against float precision
    issues
55
56      def fit(self, X: np.ndarray):
57          """Fits K-Means model via Lloyd's Expectation-Maximization
    algorithm."""
58          X = np.asarray(X, dtype=np.float64)
59          N, d = X.shape
60
61          # Initialize centroids
62          if self.init == 'k-means++':
63              self.cluster_centers_ = self._kmeans_plus_plus_init(X)
64          else:
65              rand_indices = np.random.choice(N, self.n_clusters, replace=
    False)
66              self.cluster_centers_ = X[rand_indices].copy()
67
68          for iteration in range(self.max_iter):
69              # --- Expectation Step: Assign points to nearest centroid ---
70              dists_sq = self._compute_squared_distances(X, self.
    cluster_centers_)
71              self.labels_ = np.argmin(dists_sq, axis=1)
72
73              # --- Maximization Step: Recompute centroids as cluster means
    ---
74              new_centroids = np.zeros_like(self.cluster_centers_)
75              for k in range(self.n_clusters):
76                  cluster_mask = (self.labels_ == k)
77                  if np.any(cluster_mask):
78                      new_centroids[k] = np.mean(X[cluster_mask], axis=0)
79                  else:
80                      # Handle empty cluster: reinitialize to random point
81                      new_centroids[k] = X[np.random.choice(N)]
82
83              # Check centroid convergence threshold
84              centroid_shift = np.sum((new_centroids - self.cluster_centers_)
    **2)
85              self.cluster_centers_ = new_centroids
86
87              if centroid_shift < self.tol:
88                  break
89
90          # Final WCSS calculation
91          final_dists_sq = self._compute_squared_distances(X, self.
    cluster_centers_)
92          self.inertia_ = np.sum(np.min(final_dists_sq, axis=1))
93          return self
94
```

```
    def predict(self, X: np.ndarray) -> np.ndarray:
        """Assigns new query vectors to closest fitted centroid."""
        X = np.asarray(X, dtype=np.float64)
        dists_sq = self._compute_squared_distances(X, self.cluster_centers_
    )
        return np.argmin(dists_sq, axis=1)


class KMedoidsEngine:
    """
    Production-grade K-Medoids Clustering Engine (PAM Algorithm).
    Optimizes arbitrary pairwise dissimilarity matrices with medoids
    constrained to dataset instances.
    """
    def __init__(self, n_clusters: int = 3, max_iter: int = 100, metric:
    str = 'euclidean'):
        self.n_clusters = n_clusters
        self.max_iter = max_iter
        self.metric = metric

        self.medoid_indices_ = None # Indices of selected medoids (K,)
        self.labels_ = None         # Cluster assignments (N,)
        self.cost_ = None           # Total distance cost scalar

    def _compute_pairwise_distances(self, X: np.ndarray) -> np.ndarray:
        """Computes full symmetric pairwise distance matrix D (N x N)."""
        N = X.shape[0]
        if self.metric == 'euclidean':
            X_sq = np.sum(X**2, axis=1, keepdims=True)
            dists_sq = X_sq + X_sq.T - 2.0 * (X @ X.T)
            return np.sqrt(np.maximum(dists_sq, 0.0))
        elif self.metric == 'manhattan':
            return np.sum(np.abs(X[:, np.newaxis, :] - X[np.newaxis, :, :])
    , axis=2)
        else:
            raise ValueError(f"Unsupported metric: {self.metric}")

    def fit(self, X: np.ndarray):
        """Fits K-Medoids via PAM BUILD and SWAP phases."""
        X = np.asarray(X, dtype=np.float64)
        N = X.shape[0]
        D = self._compute_pairwise_distances(X)

        # --- BUILD PHASE: Greedy Medoid Initialization ---
        self.medoid_indices_ = []
        # First medoid minimizes total distance to all other points
        first_medoid = np.argmin(np.sum(D, axis=1))
        self.medoid_indices_.append(first_medoid)

        for _ in range(1, self.n_clusters):
            # Distance to nearest existing medoid
            min_dists = np.min(D[:, self.medoid_indices_], axis=1)
            best_gain = -1.0
            next_medoid = -1

            for candidate in range(N):
                if candidate not in self.medoid_indices_:
                    # Reduction in distance if candidate becomes medoid
                    gain = np.sum(np.maximum(0.0, min_dists - D[:,
    candidate]))
```

```
                    if gain > best_gain:
                        best_gain = gain
                        next_medoid = candidate

            self.medoid_indices_.append(next_medoid)

        self.medoid_indices_ = np.array(self.medoid_indices_)

        # --- SWAP PHASE: Iterative Medoid Refinement ---
        for iteration in range(self.max_iter):
            current_cost = np.sum(np.min(D[:, self.medoid_indices_], axis
=1))
            best_swap_cost = current_cost
            best_swap_pair = None

            for k_idx in range(self.n_clusters):
                current_medoid = self.medoid_indices_[k_idx]

                for candidate in range(N):
                    if candidate not in self.medoid_indices_:
                        # Create candidate medoid set
                        temp_medoids = self.medoid_indices_.copy()
                        temp_medoids[k_idx] = candidate

                        # Evaluate cost with proposed swap
                        temp_cost = np.sum(np.min(D[:, temp_medoids], axis
=1))

                        if temp_cost < best_swap_cost:
                            best_swap_cost = temp_cost
                            best_swap_pair = (k_idx, candidate)

            # Apply best swap if cost reduction achieved
            if best_swap_pair is not None and best_swap_cost < current_cost
:
                k_swap, candidate_swap = best_swap_pair
                self.medoid_indices_[k_swap] = candidate_swap
            else:
                break # Convergence reached

        # Final Assignment
        self.labels_ = np.argmin(D[:, self.medoid_indices_], axis=1)
        self.cost_ = np.sum(np.min(D[:, self.medoid_indices_], axis=1))
        return self

    def predict(self, X_query: np.ndarray, X_train: np.ndarray) -> np.
ndarray:
        """Assigns query vectors to closest fitted medoid."""
        medoid_vectors = X_train[self.medoid_indices_]
        if self.metric == 'euclidean':
            X_sq = np.sum(X_query**2, axis=1, keepdims=True)
            M_sq = np.sum(medoid_vectors**2, axis=1, keepdims=True).T
            dists = np.sqrt(np.maximum(X_sq + M_sq - 2.0 * (X_query @
medoid_vectors.T), 0.0))
        elif self.metric == 'manhattan':
            dists = np.sum(np.abs(X_query[:, np.newaxis, :] -
medoid_vectors[np.newaxis, :, :]), axis=2)
        return np.argmin(dists, axis=1)


```

```python
if __name__ == "__main__":
    np.random.seed(42)

    # Generate Synthetic 3-Cluster Dataset with Outliers
    N_per_cluster = 100
    c1 = np.random.multivariate_normal([-3, -3], [[0.5, 0], [0, 0.5]], N_per_cluster)
    c2 = np.random.multivariate_normal([3, 3], [[0.5, 0], [0, 0.5]], N_per_cluster)
    c3 = np.random.multivariate_normal([0, 4], [[0.5, 0], [0, 0.5]], N_per_cluster)

    # Add extreme outliers
    outliers = np.array([[20, 20], [-18, 15]])

    X_raw = np.vstack([c1, c2, c3, outliers])

    # 1. Evaluate K-Means
    kmeans = KMeansEngine(n_clusters=3, init='k-means++')
    kmeans.fit(X_raw)

    print("=== K-MEANS ENGINE INITIALIZATION SUCCESSFUL ===")
    print(f"Final WCSS (Inertia) : {kmeans.inertia_:.4f}")
    print(f"Computed Centroids:\n{kmeans.cluster_centers_}\n")

    # 2. Evaluate K-Medoids (PAM)
    kmedoids = KMedoidsEngine(n_clusters=3, metric='manhattan')
    kmedoids.fit(X_raw)

    print("=== K-MEDOIDS (PAM) ENGINE INITIALIZATION SUCCESSFUL ===")
    print(f"Final Dissimilarity Cost : {kmedoids.cost_:.4f}")
    print(f"Selected Medoid Indices : {kmedoids.medoid_indices_}")
    print(f"Selected Medoid Vectors:\n{X_raw[kmedoids.medoid_indices_]}")
```

Listing 1: Vectorized NumPy implementation of K-Means and K-Medoids engines.